

Hybrid Computing Technology

1

ASIC-Based and TPU Computing Technologies

ผศ. อัครินทร์ คุณกิตติ

ASST. PROF. AKHARIN KHUNKITTI

FACULTY OF INFORMATION TECHNOLOGY
KMITL



Topics

2

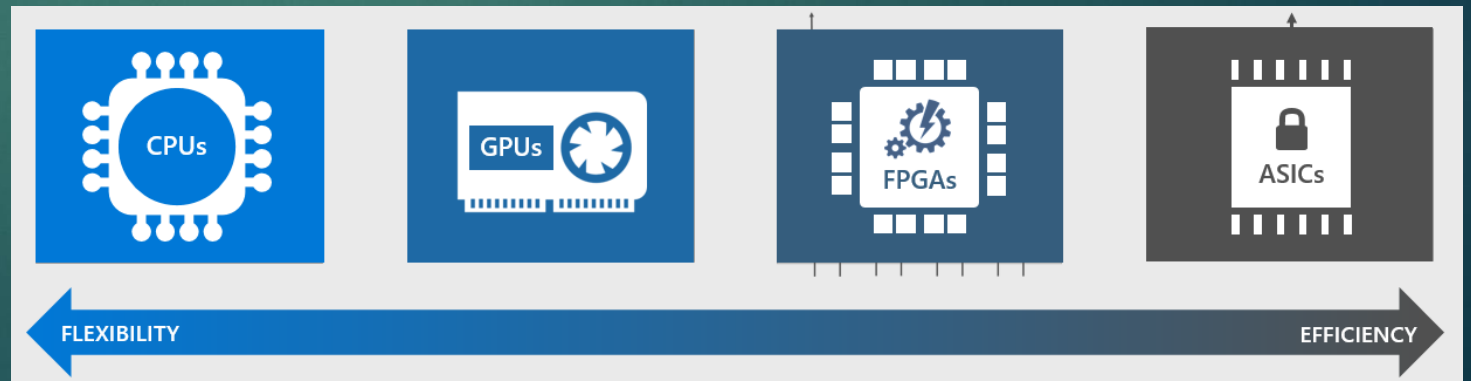
- ▶ Introduction to ASIC
- ▶ ASIC Computing
- ▶ ASIC Code Development
- ▶ AI/ML Specific Accelerators
- ▶ TPU Computing
- ▶ Conclusion



Introduction to ASIC

3

- ▶ Application-Specific Integrated Circuit (ASIC)
- ▶ An integrated circuit (IC) chip customized for a particular use, rather than intended for general-purpose use.
- ▶ Designed for Specific Task
- ▶ Fix Design => Can not change
- ▶ Contrast to FPGA => Programmable
- ▶ Can be Optimized for
 - ▶ Functions
 - ▶ Speed
 - ▶ Power



ASIC vs FPGA

4

any silicon

FPGA

ASIC

Time to Market

Fast

Slow

NRE

Low

High

Design Flow

Simple

Complex

Unit Cost

High

Low

Performance

Medium

High

Power Consumption

High

Low

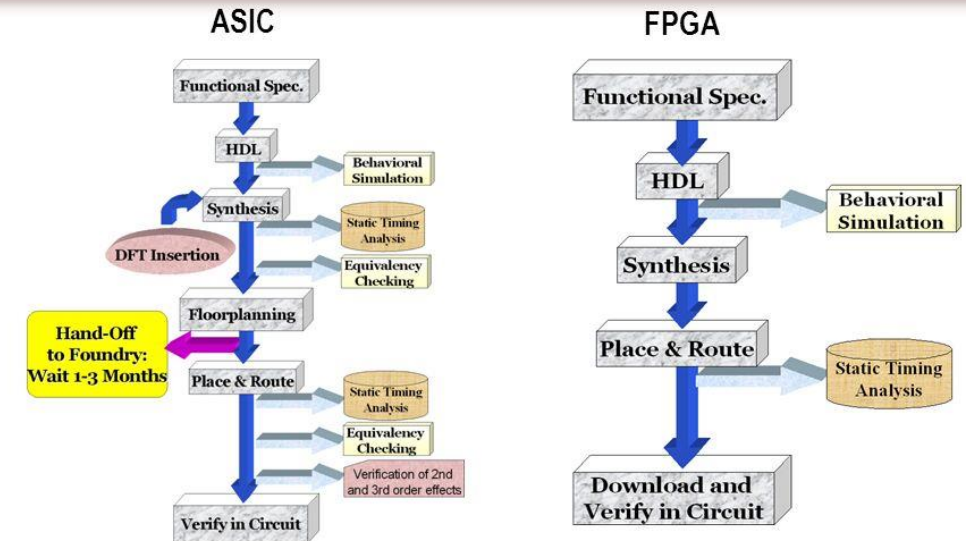
Unit Size

Medium

Low

AnySilicon.com

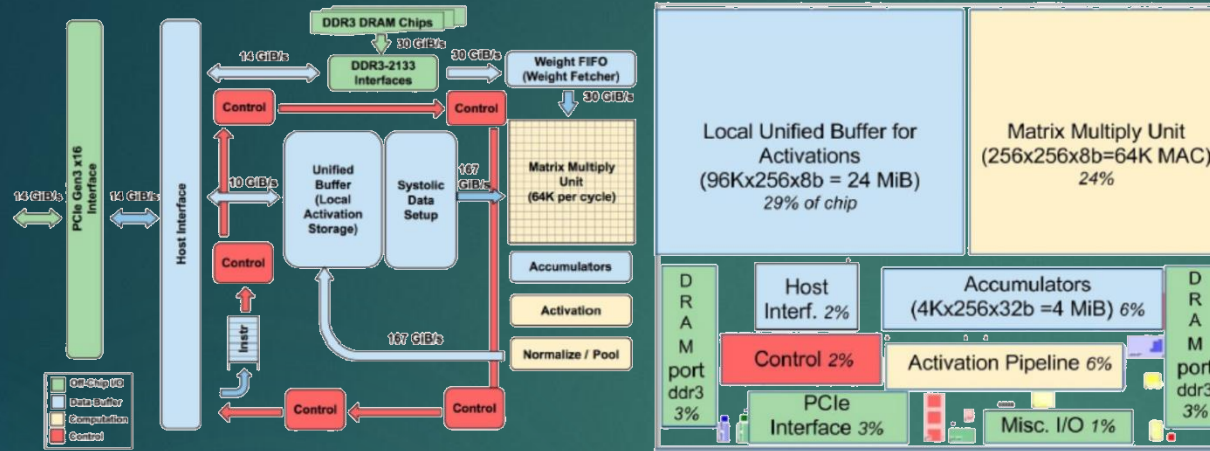
Design Flow Comparison



NRE stands for Non Recurring Engineering costs

ASIC Example

The representative of the cloud AI chip :
TPU from Google



TPU Block Diagram. The main computation part is the yellow Matrix Multiply unit in the upper right hand corner. Its inputs are the blue Weight FIFO and the blue Unified Buffer (UB) and its output is the blue Accumulators (Acc). The yellow Activation Unit performs the nonlinear functions on the Acc, which go to the UB.

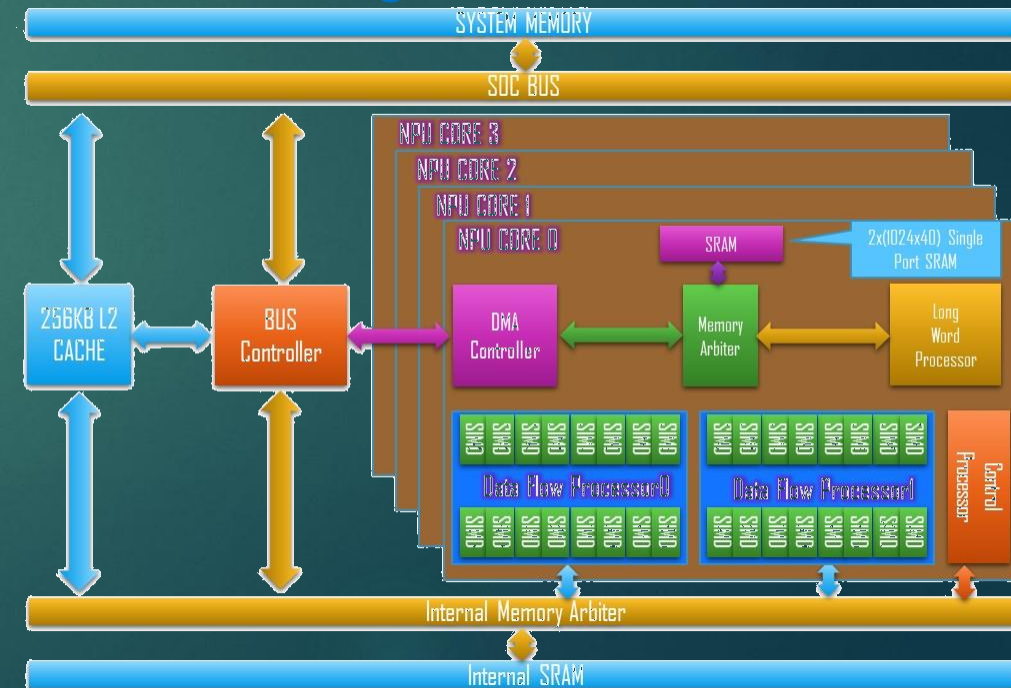
Floor Plan of TPU die. The light (blue) data buffers are 37% of the die, the light (yellow) compute is 30%, the medium (green) I/O is 10%, and the dark (red) control is just 2%. Control is much larger (and much more difficult to design) in a CPU or GPU

- ✓ The main computing power comes from the MAC matrix.
- ✓ MAC matrix, systolic array, SIMD, cluster, heterogeneous processing & communication engines and so on are referring to essentially similar parallel processing paradigm
- ✓ Architectural wise, the parallelism are expressed by the deep learning algorithm itself
- ✓ A good architecture and a poor architecture could have several times gap in terms of performance, but not dozens of times.

5

Cloud Side	Edge Side
64K MACs	512 - 4K MACs
Floating point	Fixed point
75W	2W
20 DDR chips	1-2 DDR chips

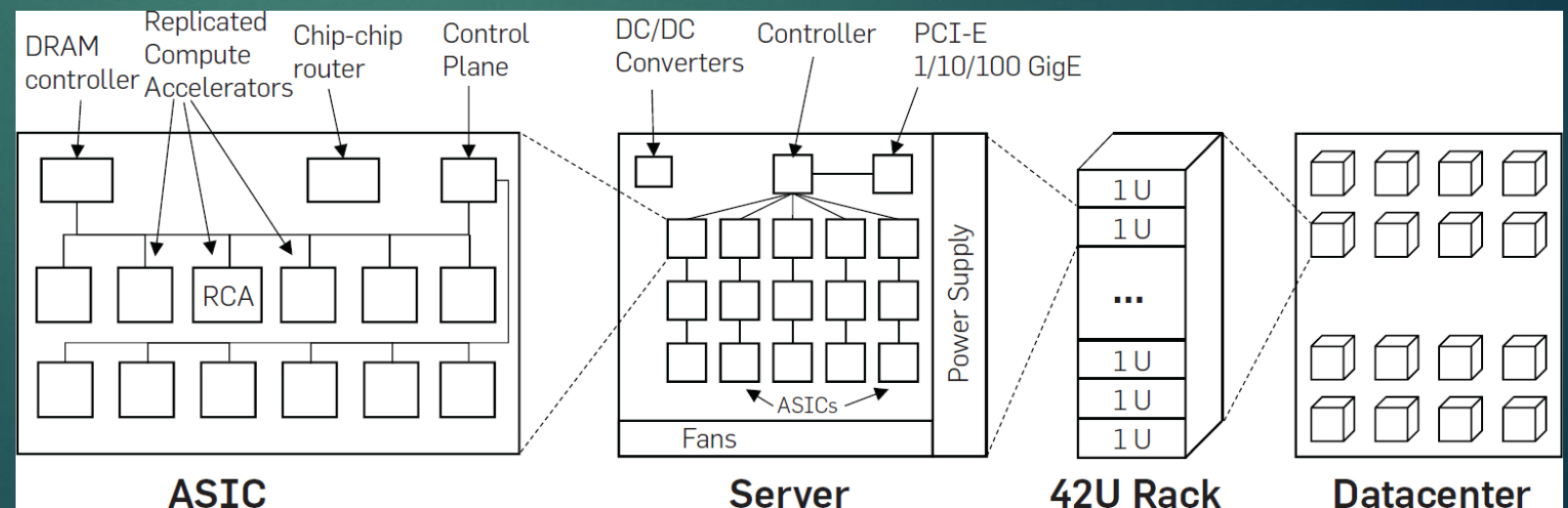
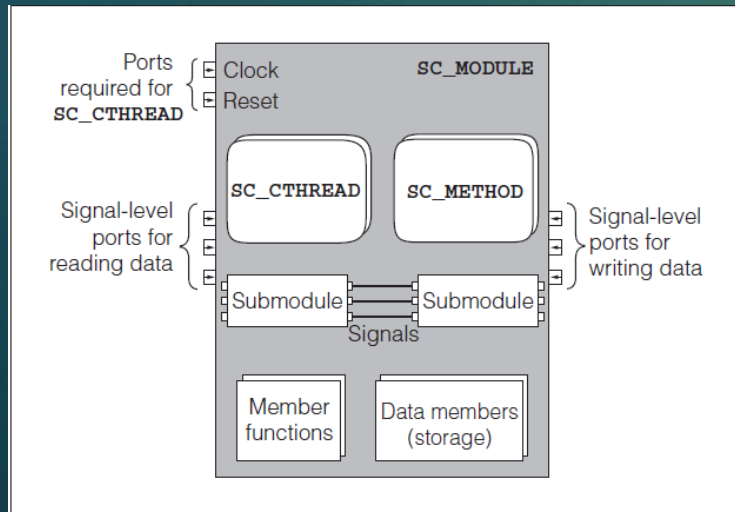
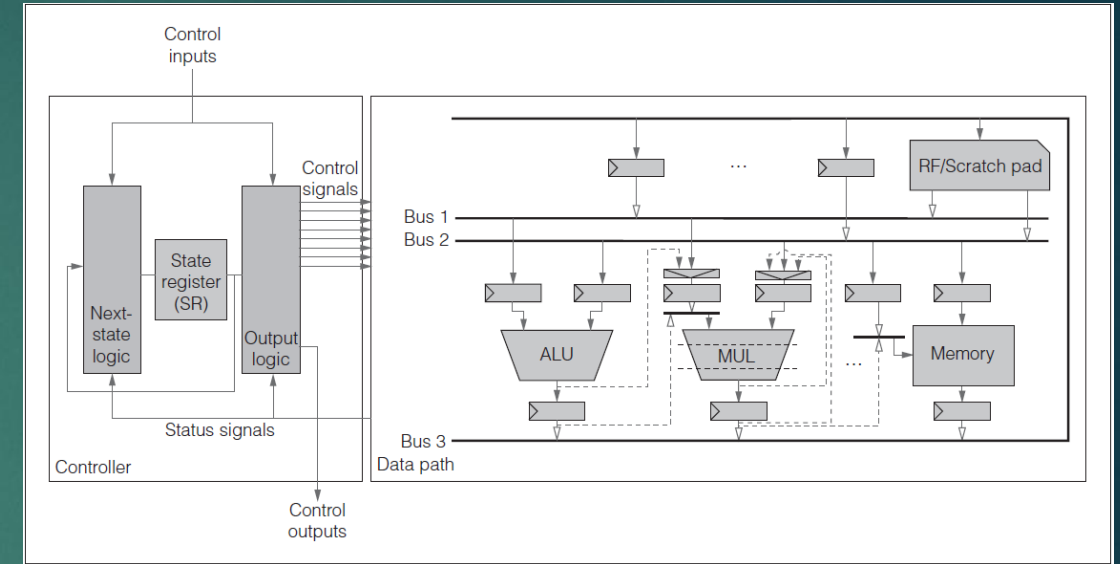
The representative of the edge AI chip :
Starlight from VimicroAI



ASIC Computing

6

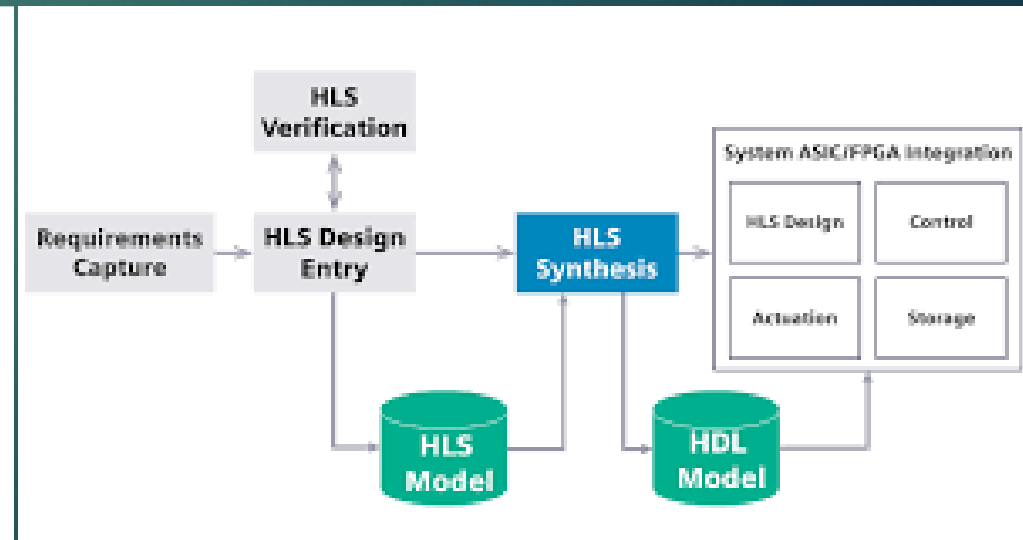
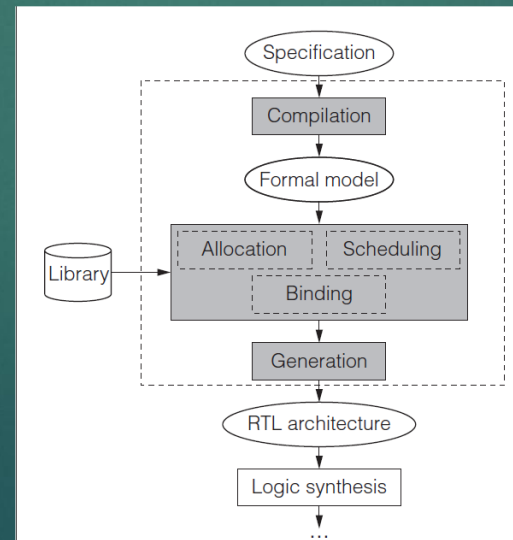
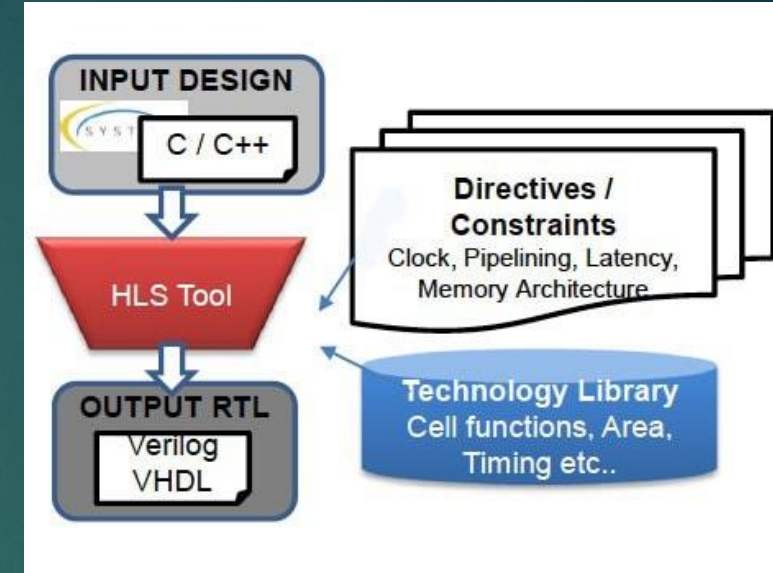
- ▶ ASIC => Fix Computing Logics
- ▶ ASIC Cards => Accelerators
- ▶ Host => Controller
 - ▶ Program calls ASIC Logics



ASIC Code Development

7

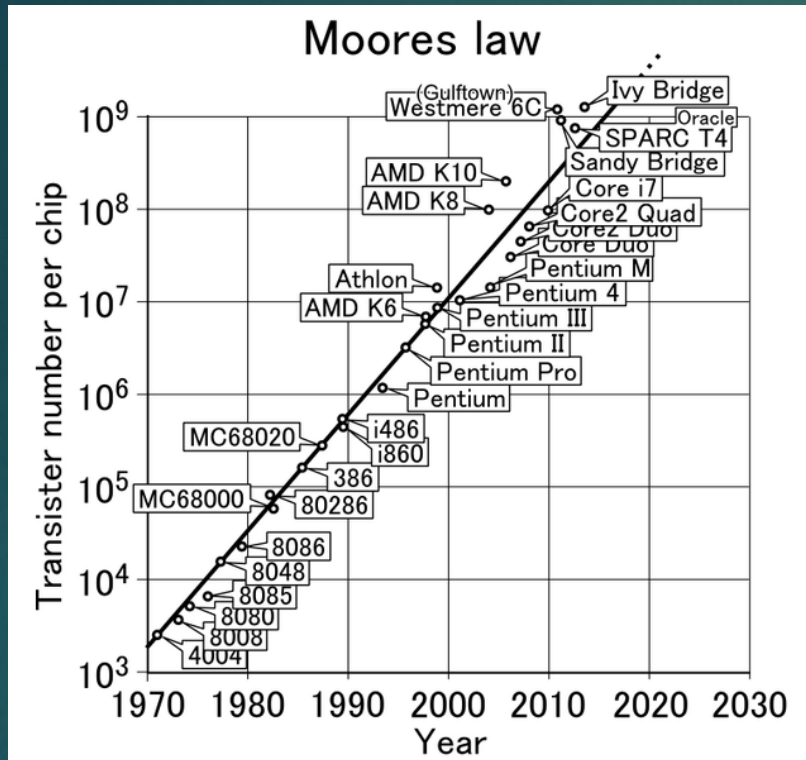
- ▶ HLS – High Level Synthesis
 - ▶ High Level Programming Language
 - ▶ Source Code for Specific Tasks/Functions
- ▶ Compiler / Tools
 - ▶ Hardware/Logic Codes
 - ▶ HDL
- ▶ Manufacturing
 - ▶ ASIC Chips
 - ▶ ASIC Cards
- ▶ Host Control/Executing



AI/ML Specific Accelerators

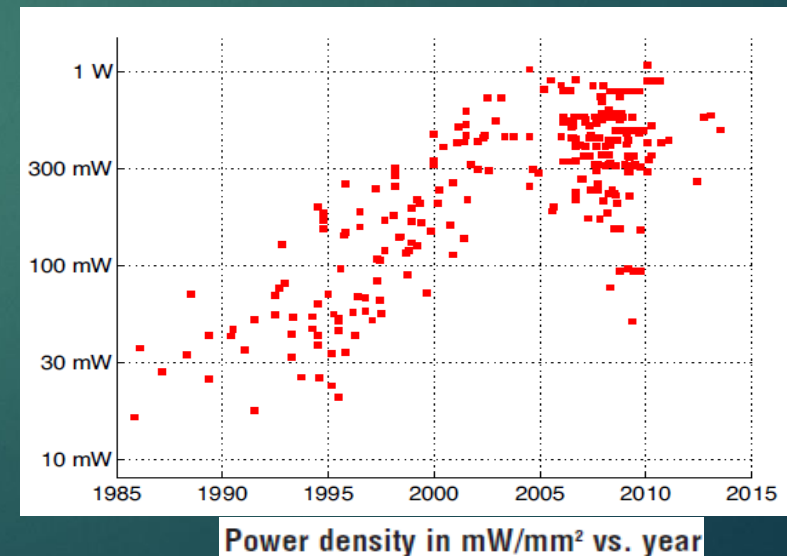
Inspiration – Technology Trends

9



- ▶ Moore's Law had many lives
 - ▶ 2004: The end of Moore's Law?
 - ▶ 2015: Beyond Moore's Law
 - ▶ 2016: After Moore's Law
 - ▶ 2017: A new way to extend Moore's Law

- ... but Dennard's Scaling has stopped
 - On-chip power density cannot continue to grow



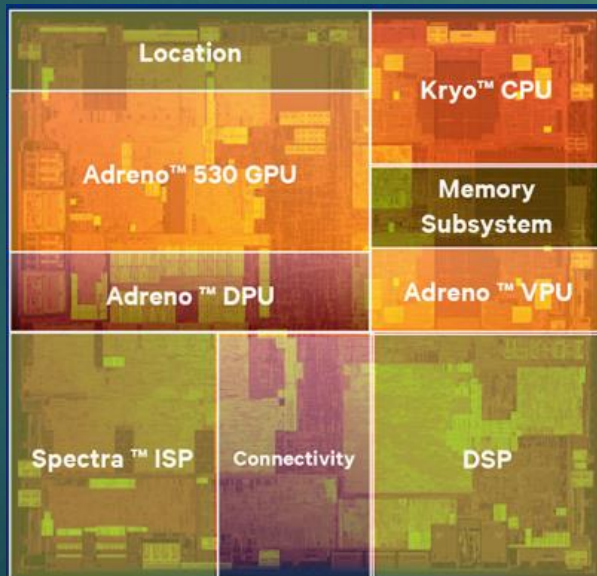
Source: Horowitz et al., 2014

Emerging Computing Platforms for AI

10

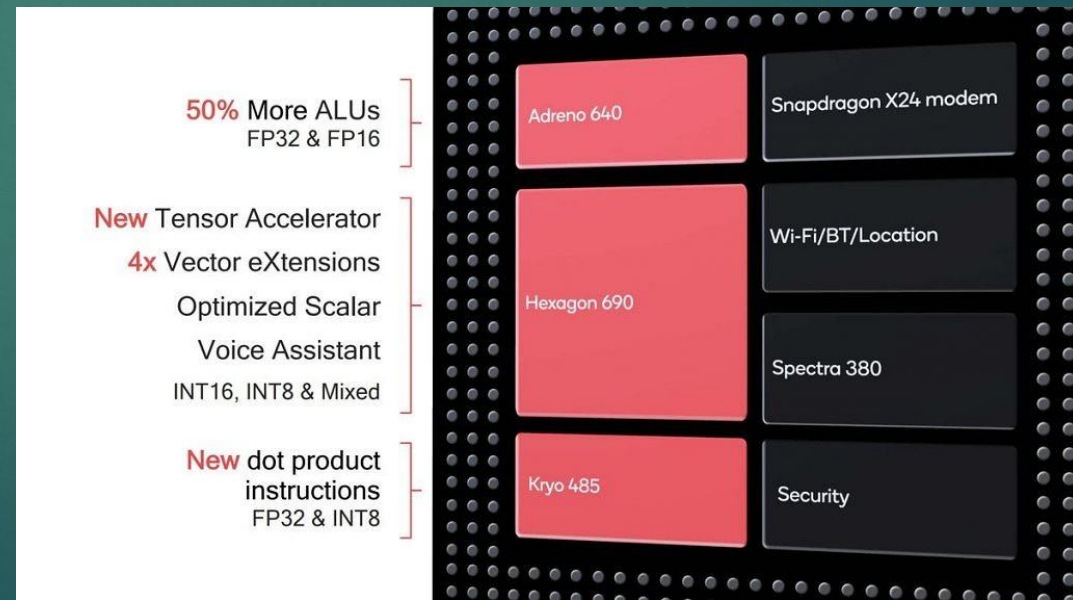
▶ Heterogeneous Multi-Core Systems

- ▶ Mix of processor cores and specialized hardware accelerators yields more energy-efficient computing
- ▶ The **approach to AI** is getting **heterogeneous**
 - ▶ DSP, GPU, CPU, FPGA, custom hardware...
 - ▶ ... whatever is the best piece of hardware to handle an AI task in the most power efficient way



Source: Qualcomm

Samsung Snapdragon 820, 2016



Source: Qualcomm

Samsung Snapdragon 855, 2019

Accelerator

11

- ▶ A **special-purpose hardware** that is optimized to perform a **dedicated function(s)** as part of a **general-purpose computational system**
 - ▶ While being part of a larger system, it spans a scale from being closely integrated with a general purpose processor to being attached to a computing system
 - ▶ Increasingly accelerators are migrating into the chip, thus leading to the rise of heterogeneous multi-core SoC architectures"
- ▶ Implemented as
 - ▶ Application Specific Integrated Circuit (**ASIC**)
 - ▶ Field-Programmable Gate Array (**FPGA**)

High-Level Synthesis

12

hls4ml

- RF
- Precision
- Latency/Resources

HLS
Directives

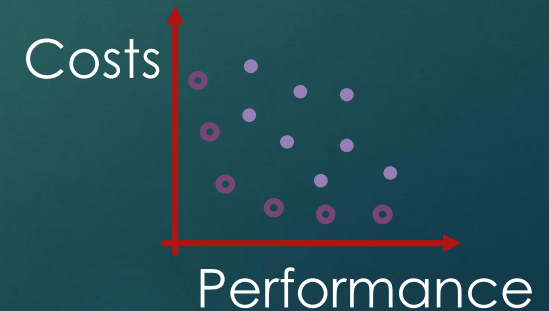
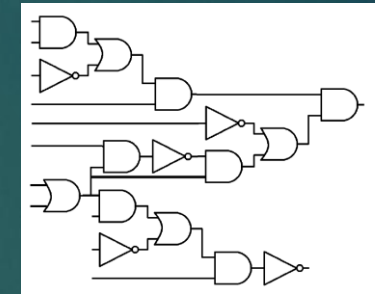
```
inline bar 'ON'  
unroll 11 factor=10  
bundle b1=A,B, b2=C
```

Software
Specification

```
void foo(int A[10],  
        int B[10], int C[10])  
{  
    int i = 0;  
    11: for (; i < 10; i++) {  
        A[i] = B[i] * i;  
    }  
    i = 0;  
    12: for (; i < 10; i++) {  
        B[i] = A[i] * B[i];  
        C[i] = B[i] / 10;  
    }  
    bar(A, B);  
}
```

HLS

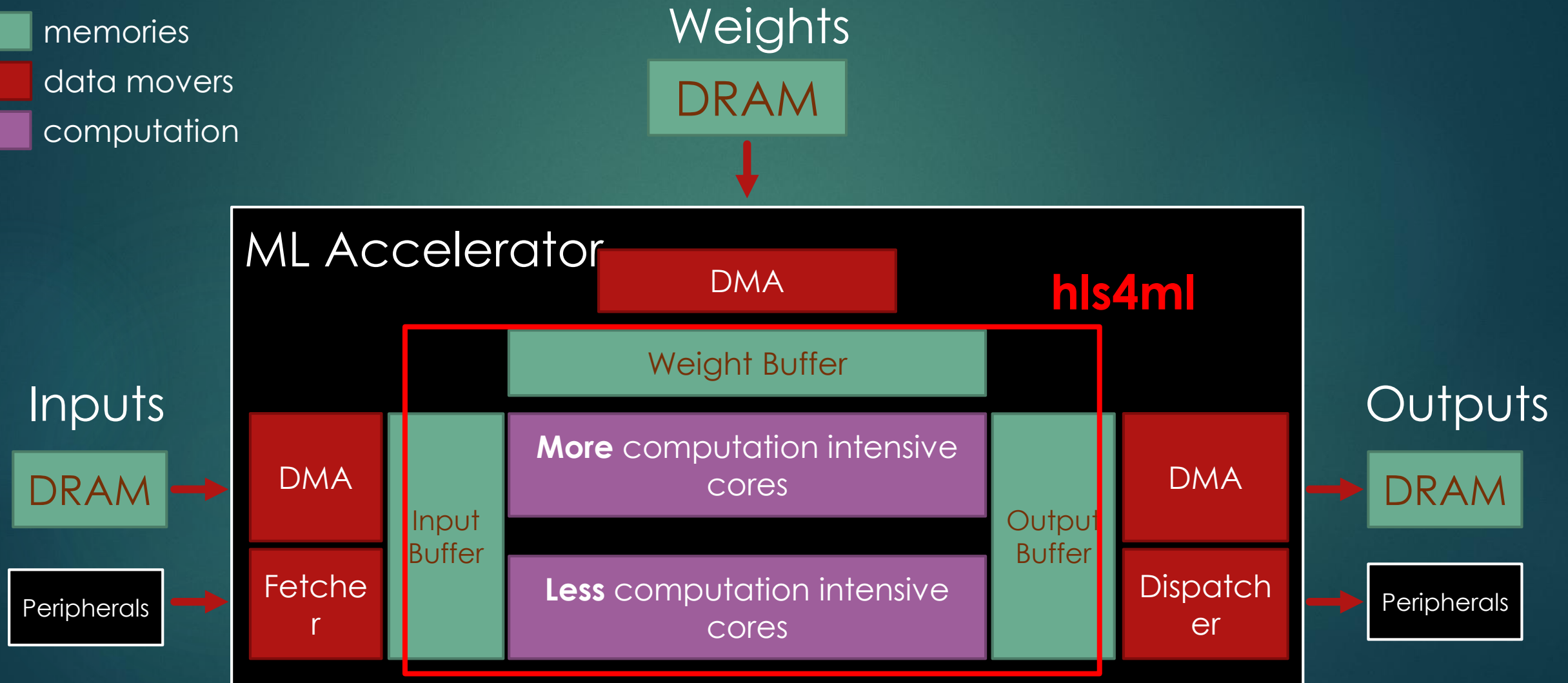
Hardware
Implementation(s)



Architecture of a ML Accelerator

13

- memories
- data movers
- computation



Reducing Bit-Precision

14

- ▶ Arithmetic

- ▶ Floating Point

- ▶ FP64, FP32, FP16, FP11 ...

- ▶ Fixed Point (Linear Quantization)

- ✓ A lot of freedom

- ✓ “Essentially” integer arithmetic

- ✗ Overflow and underflow problems

- ▶ Binary quantization

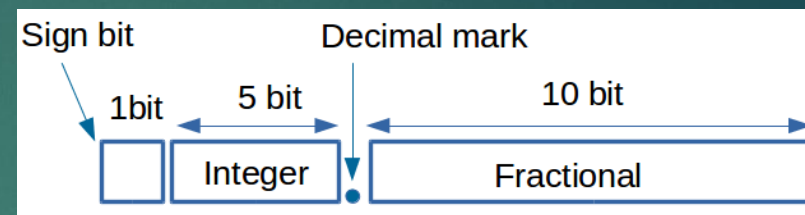
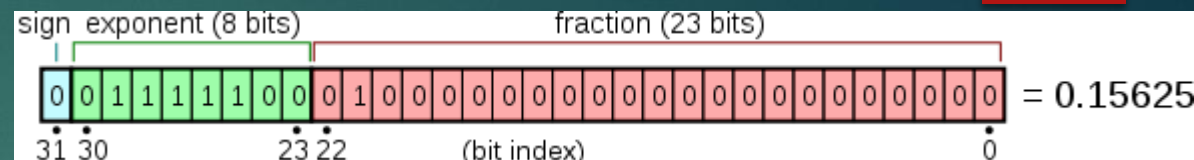
- ▶ Linear reduction in **memory footprint**

- ▶ Reduce the amount of data transfer

- ▶ Model may fit local buffers (on-chip)

- ▶ Reduction of the **arithmetic logic**

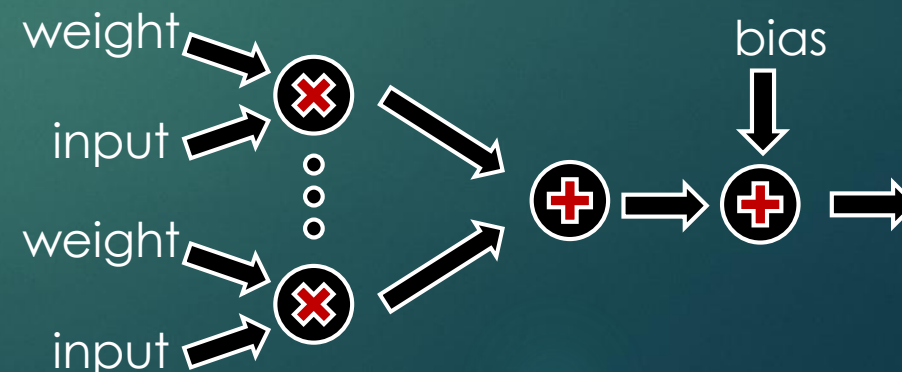
- ▶ Improve area, power, latency



$$W_i = \dots * \text{bits}$$

$$A_i = \dots * \text{bits}$$

$$A_{i+1} = \dots * \text{bits}$$



Hardware Architecture

15

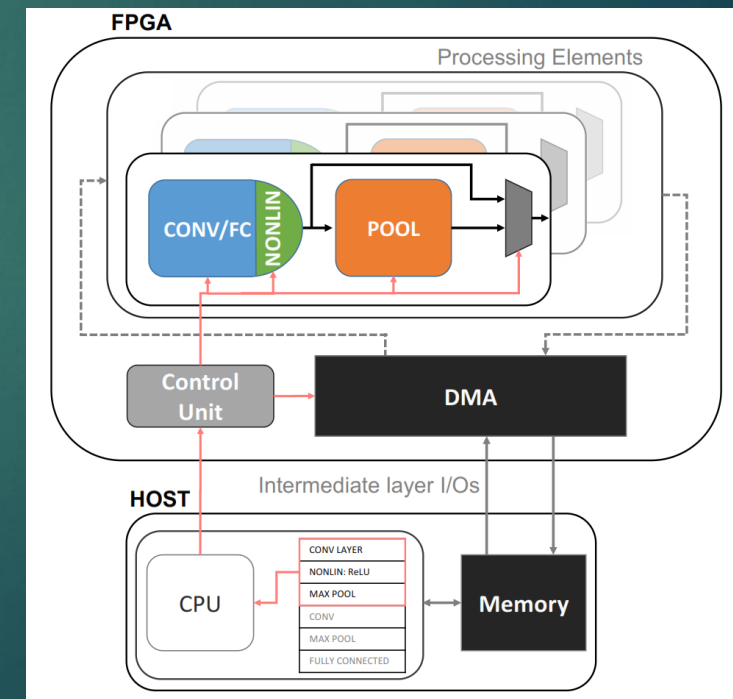
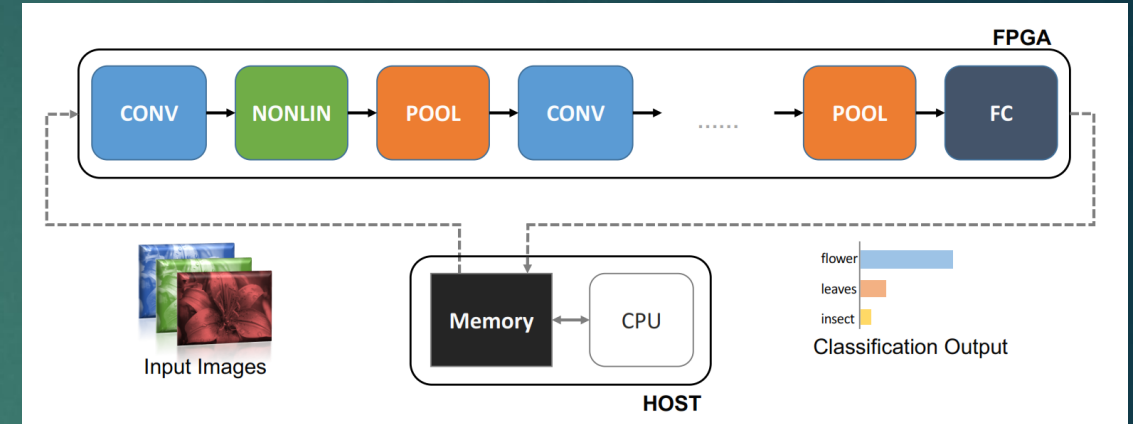
▶ Streaming

- ▶ One distinct hardware block for each CNN layer, where each block is optimized separately to exploit the inner parallelism of the layer

- ▶ Weights: on-chip or off-chip
- ▶ Controller: software or hardware

▶ Single Computation Engine

- ▶ Single computation engine that executes the CNN layers sequentially
 - ▶ Software controller of the scheduling of operations (may be inefficient)
 - ▶ Flexibility and reusability over customization
 - ▶ “One-size-fits-all” approach has higher performance on NN with a uniform structure



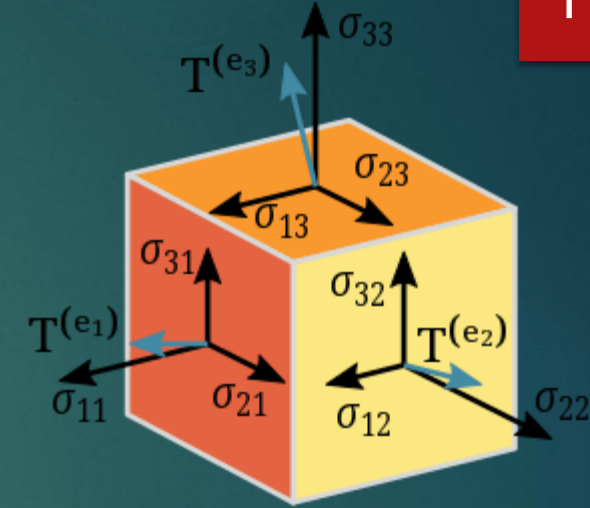
Tensor Processing Unit (TPU) Computing

BY GOOGLE

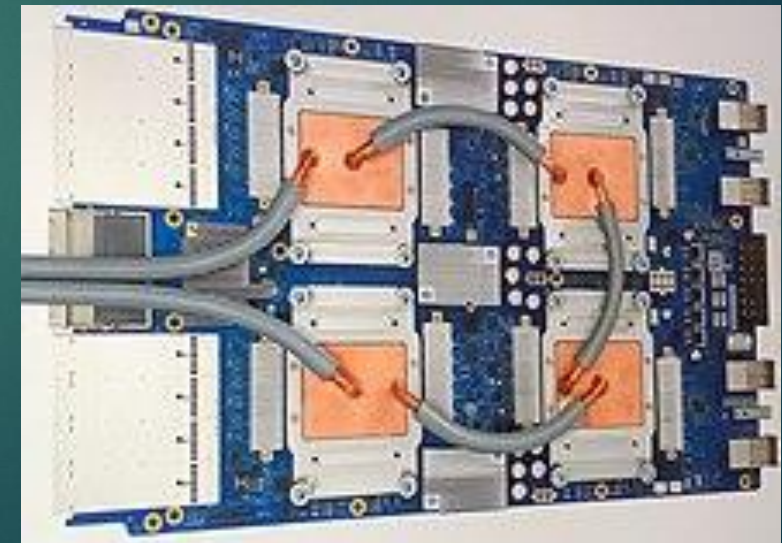
TPU Computing

17

- ▶ Tensor Processing Unit (TPU)
- ▶ Tensor $\Rightarrow$ Algebra/Number
- ▶ AI Accelerator ASIC
 - ▶ Developed by Google
 - ▶ Specifically for neural network machine learning



Chip	TPUv1	TPUv2	TPUv3
Announced	2016	May-17	May-18
Access	Internal-Only	Service Beta	Undisclosed
Introduction	2015	Feb 2018	Undisclosed
Process	28nm	20nm est.	16/12nm est.
Die Size	~300mm ²	Undisclosed	Undisclosed
TOPS	92 / 23	45	90
Matrix Input	INT8 / INT16	bfloat16	bfloat16
Memory	8GB DDR3	16GB HBM	32GB HBM
CPU Interface	PCIe 3.0 x16	PCIe 3.0 x8	PCIe 3.0 x8 est.
Power Consumption	40W	200-250W est.	200W est.



Motivation

18

- ▶ 2006: only a few applications could run on custom H/W (ASIC, FPGA, GPUs) —> thus you could easily find those resources in a datacenter (due to under-utilization)
- ▶ 2013: predictions on the wide applicability of another computational paradigm called Neural Networks (NN), could double the computation demands on datacenters.
- ▶ It would be very expensive to increase the GPUs in order to satisfy those needs

Goals

19

- ▶ Proposal:
 - ▶ Design a custom ASIC for the inference phase of NN (training still happens using GPUs)
- ▶ Principles:
 - ▶ improve cost-performance by 10X compared to GPUs
 - ▶ simple design for **response time guarantees**
 - ▶ (single-thread, no prefetching, no OOO etc)
- ▶ Characteristics:
 - ▶ More like a co-processor to reduce time-to market delays
 - ▶ Host sends instructions to TPU
 - ▶ connected through PCIe I/O bus

Quantization & Datatypes

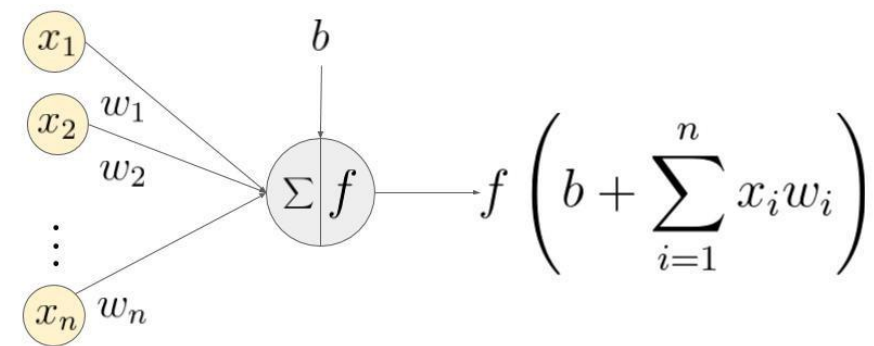
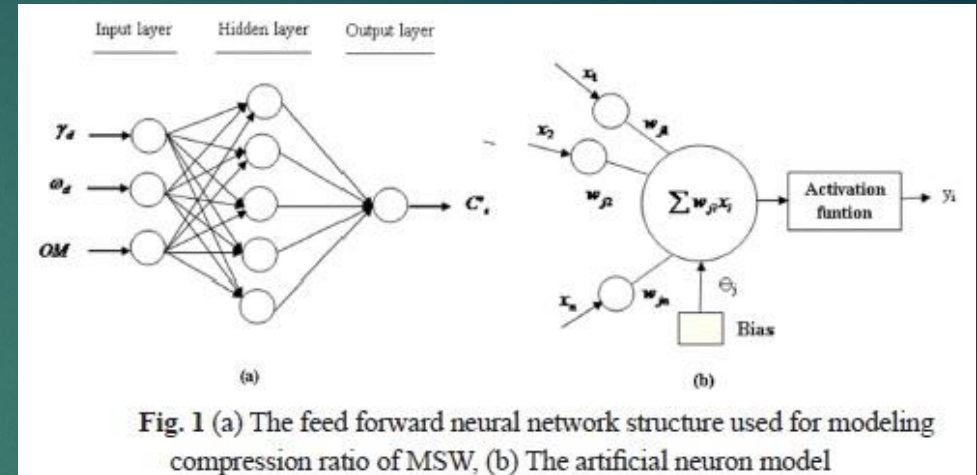
20

- ▶ Quantization:
 - ▶ IEEE FP $\rightarrow$ 8bit fixed point
 - ▶ 8b multiply $\rightarrow$ 6x energy-efficiency / 6x area
 - ▶ 8b addition $\rightarrow$ 13x energy-eff. / 38x area

Target NNs

21

- ▶ Multi-level Perceptrons
 - ▶ Fully-connected layer
- ▶ Convolutional Networks
 - ▶ Chain/DAG of Conv/Activation/Pooling/Fully-Connected
- ▶ Recursive Neural Networks
 - ▶ LSTM most popular
 - ▶ Fully-connected + vector



An example of a neuron showing the input ($x_1 - x_n$), their corresponding weights ($w_1 - w_n$), a bias (b) and the activation function f applied to the weighted sum of the inputs.

Neural Network Models

22

<i>Name</i>	<i>LOC</i>	<i>Layers</i>					<i>Nonlinear function</i>	<i>Weights</i>	<i>TPU Ops / Weight Byte</i>	<i>TPU Batch Size</i>	<i>% of Deployed TPUs in July 2016</i>
		<i>FC</i>	<i>Conv</i>	<i>Vector</i>	<i>Pool</i>	<i>Total</i>					
MLP0	100	5				5	ReLU	20M	200	200	61%
MLP1	1000	4				4	ReLU	5M	168	168	
LSTM0	1000	24		34		58	sigmoid, tanh	52M	64	64	29%
LSTM1	1500	37		19		56	sigmoid, tanh	34M	96	96	
CNN0	1000		16			16	ReLU	8M	2888	8	5%
CNN1	1000	4	72		13	89	ReLU	100M	1750	32	

Table 1. Six NN applications (two per NN type) that represent 95% of the TPU's workload. The columns are the NN name; the number of lines of code; the types and number of layers in the NN (FC is fully connected, Conv is convolution, Vector is self-explanatory, Pool is pooling, which does nonlinear downsizing on the TPU; and TPU application popularity in July 2016. One DNN is RankBrain [Cla15]; one LSTM is a subset of GNM Translate [Wu16]; one CNN is Inception; and the other CNN is DeepMind AlphaGo [Sil16][Jou15].

Main Take-aways

23

- ▶ Inference apps usually emphasize response-time over throughput since they are often user-facing.
- ▶ Due to latency limits, the K80 GPU is underutilized for inference, and is just a little faster than a Haswell CPU.
- ▶ A smaller and lower power chip, the TPU has 25X as many MACs and 3.5X as much on-chip memory as the K80 GPU.
- ▶ The TPU is about 15X - 30X faster at inference than the K80 GPU and the Haswell CPU.

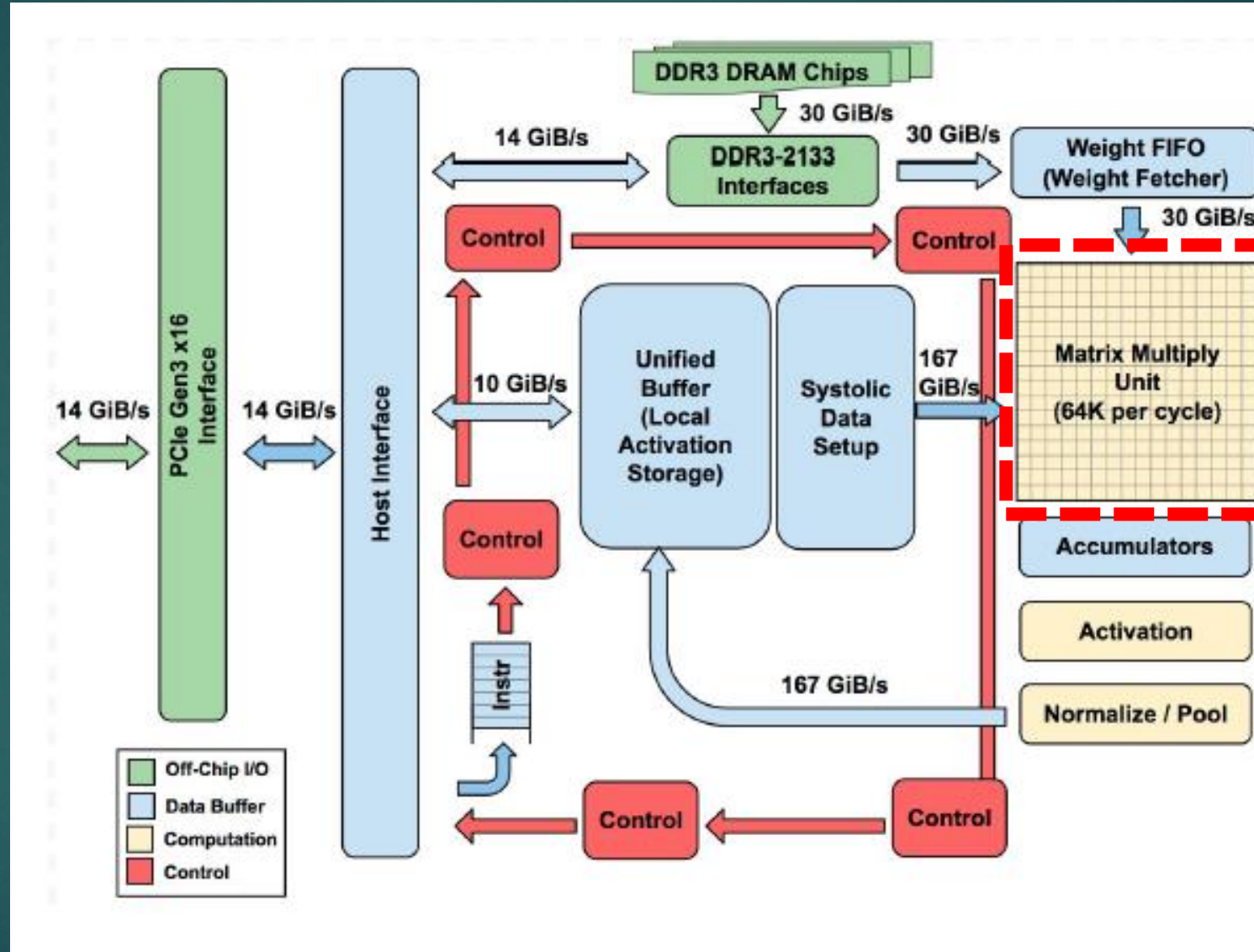
Main Take-aways #2

24

- ▶ Four of the six NN apps are memory-bandwidth limited on the TPU;
 - ▶ if the TPU were revised to have the same memory system as the K80 GPU, it would be about 30X - 50X faster than the GPU and CPU.
- ▶ The performance/Watt of the TPU is 30X - 80X that of contemporary products;
 - ▶ the revised TPU with K80 memory would be 70X - 200X better.
- ▶ While most architects have been accelerating CNNs, they represent just 5% of our datacenter workload.

TPU Architecture

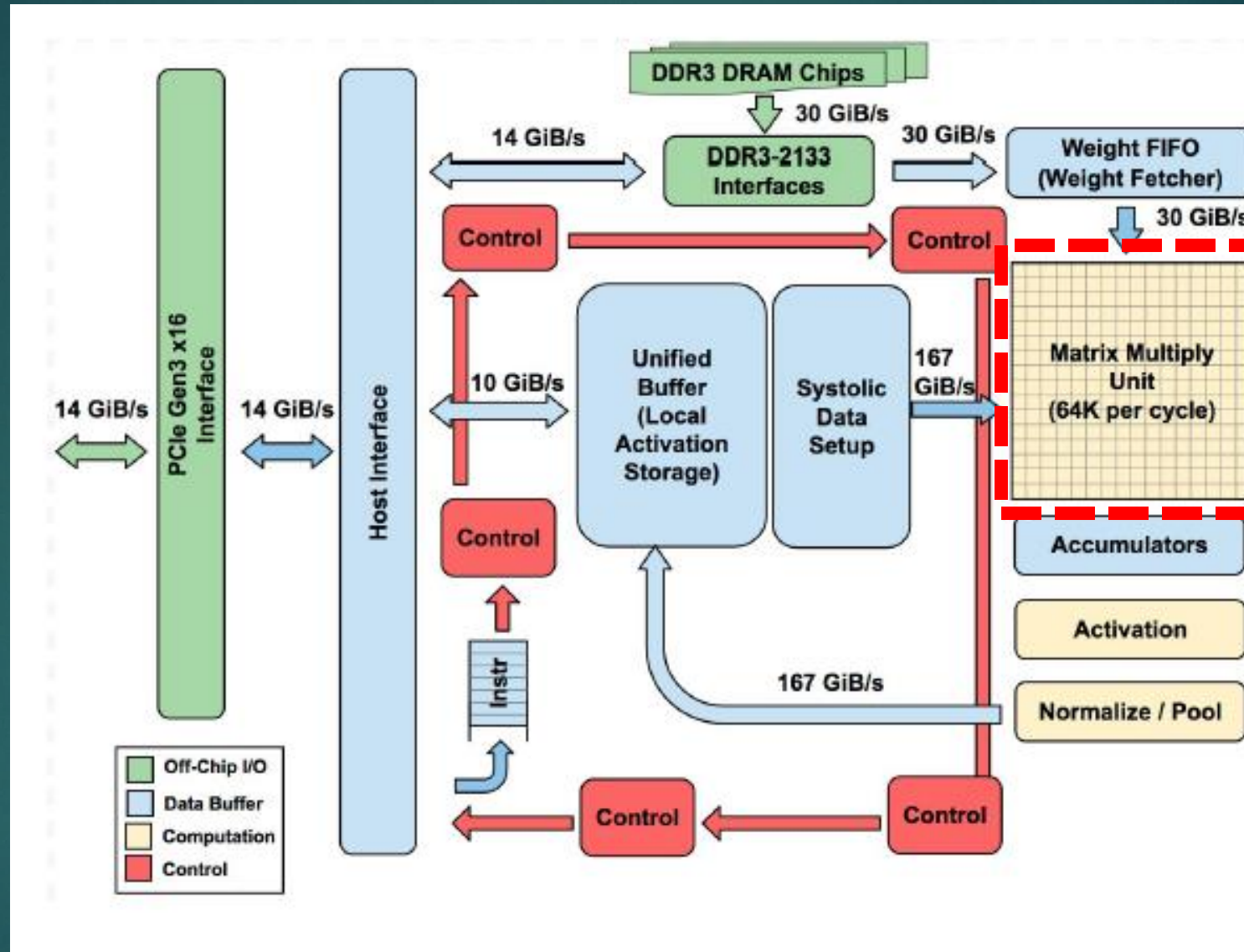
25



All internal buses are 256B wide

TPU Architecture

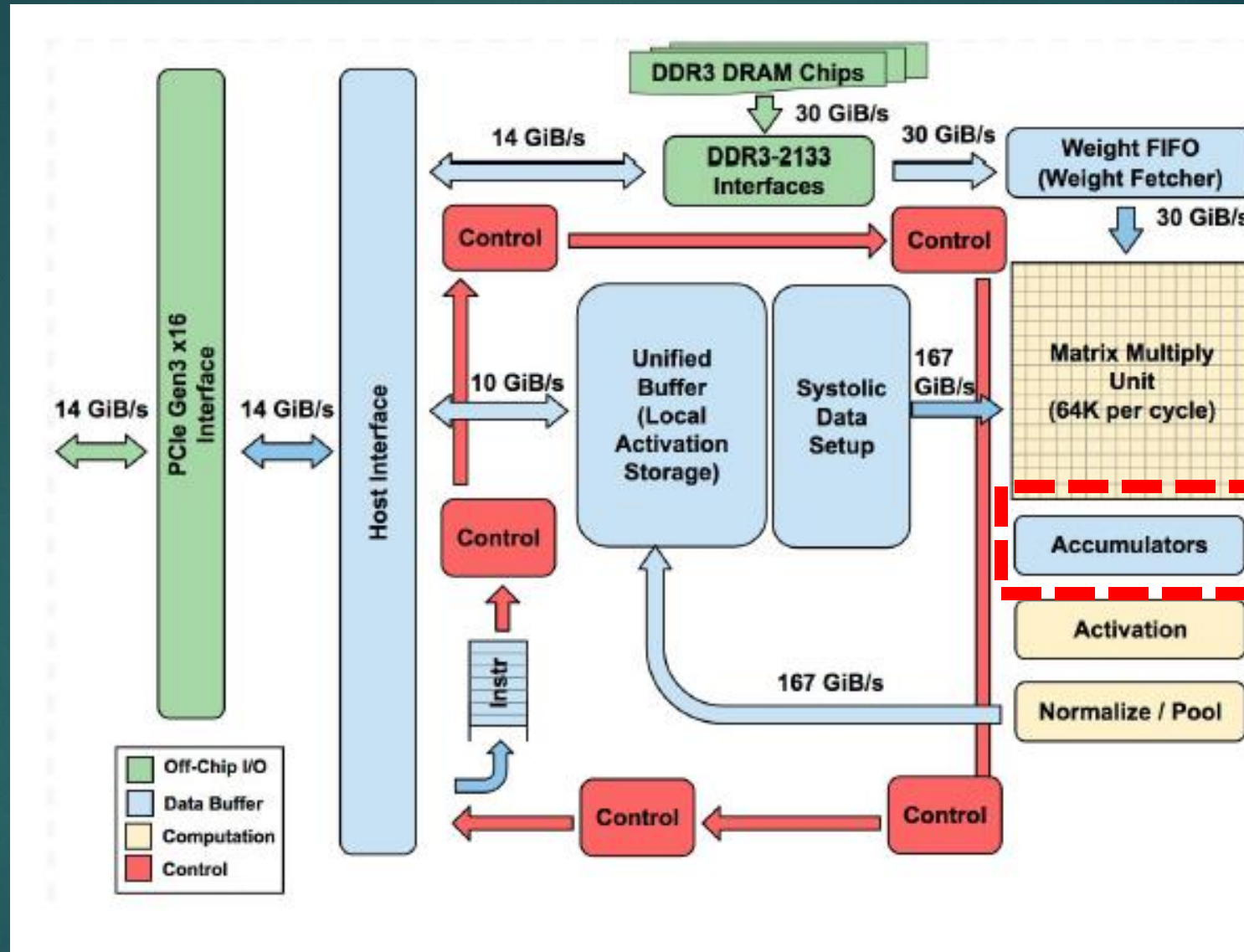
26



Main Block: MMU 256 x 256 MACs, 8b signed or unsigned ints.
256 partial sums per cycle

TPU Architecture

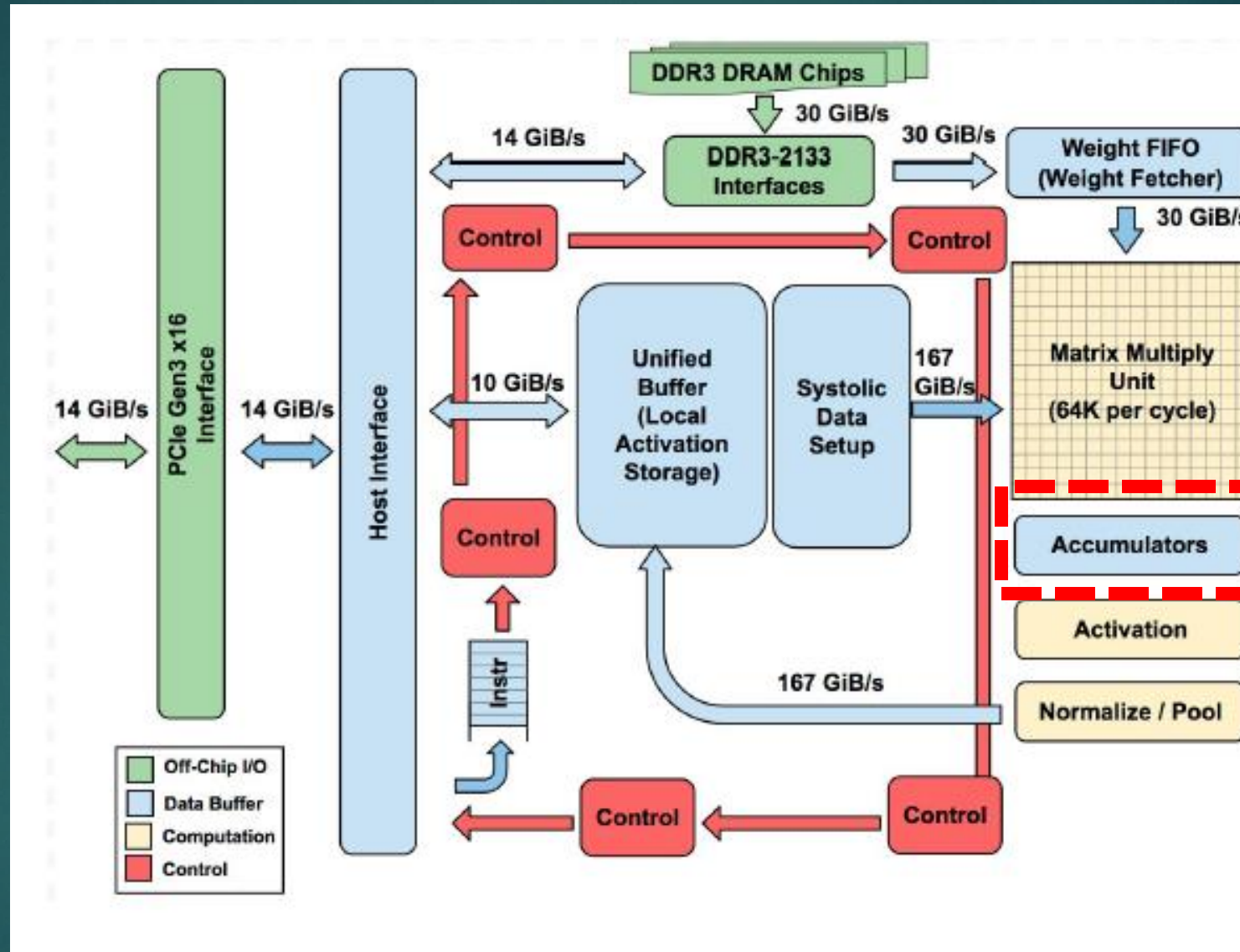
27



16bit products accumulated in the Accumulators 😊

TPU Architecture

28



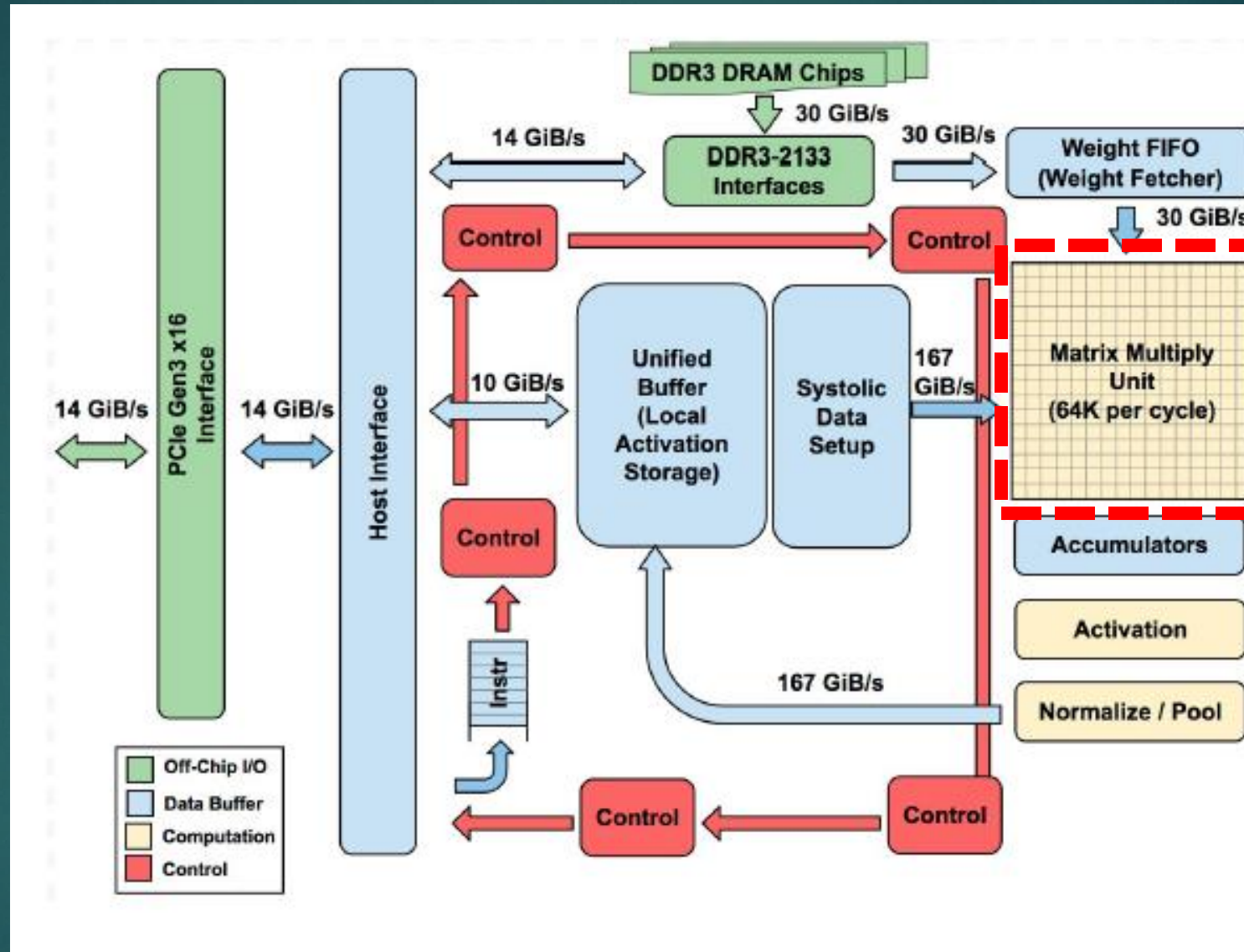
4MiB: 4K x 256 x 32b accumulators (vectors)

Why 4K vectors in the Accumulators

- ▶ Calculated for Peak performance;
 - ▶ Need ~1350
 - ▶ Rounded that up to 2048
 - ▶ Double that so that we can have double buffering:
 - ▶ Reading past results while calculating the next set

TPU Architecture

30



MMU: can do mix of 8b, or 16b
½ speed for 8b x 16b, ¼ speed for 16b x 16b

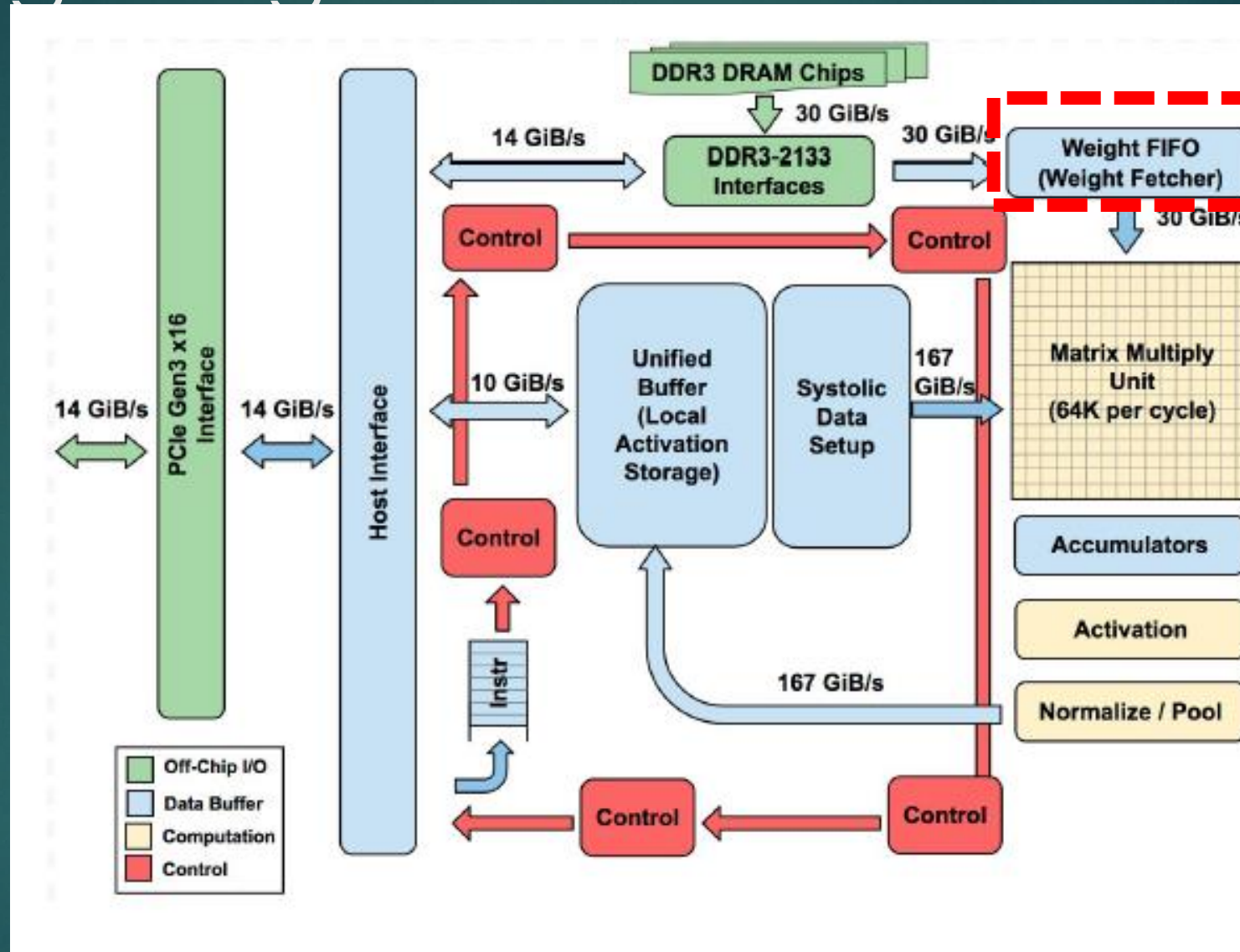
MMU contd.

31

- ▶ Contains 64KiB weights x 2
- ▶ One tile is the active the other is for double buffering
 - ▶ Loading the next while calculating with the current one
 - ▶ Needs 256 cycles to load next one
- ▶ Can do Matrix Multiply or Convolution
- ▶ Can read or write 256 values per cycle
- ▶ Targets Dense networks, Sparse later revisions

Loading Weights

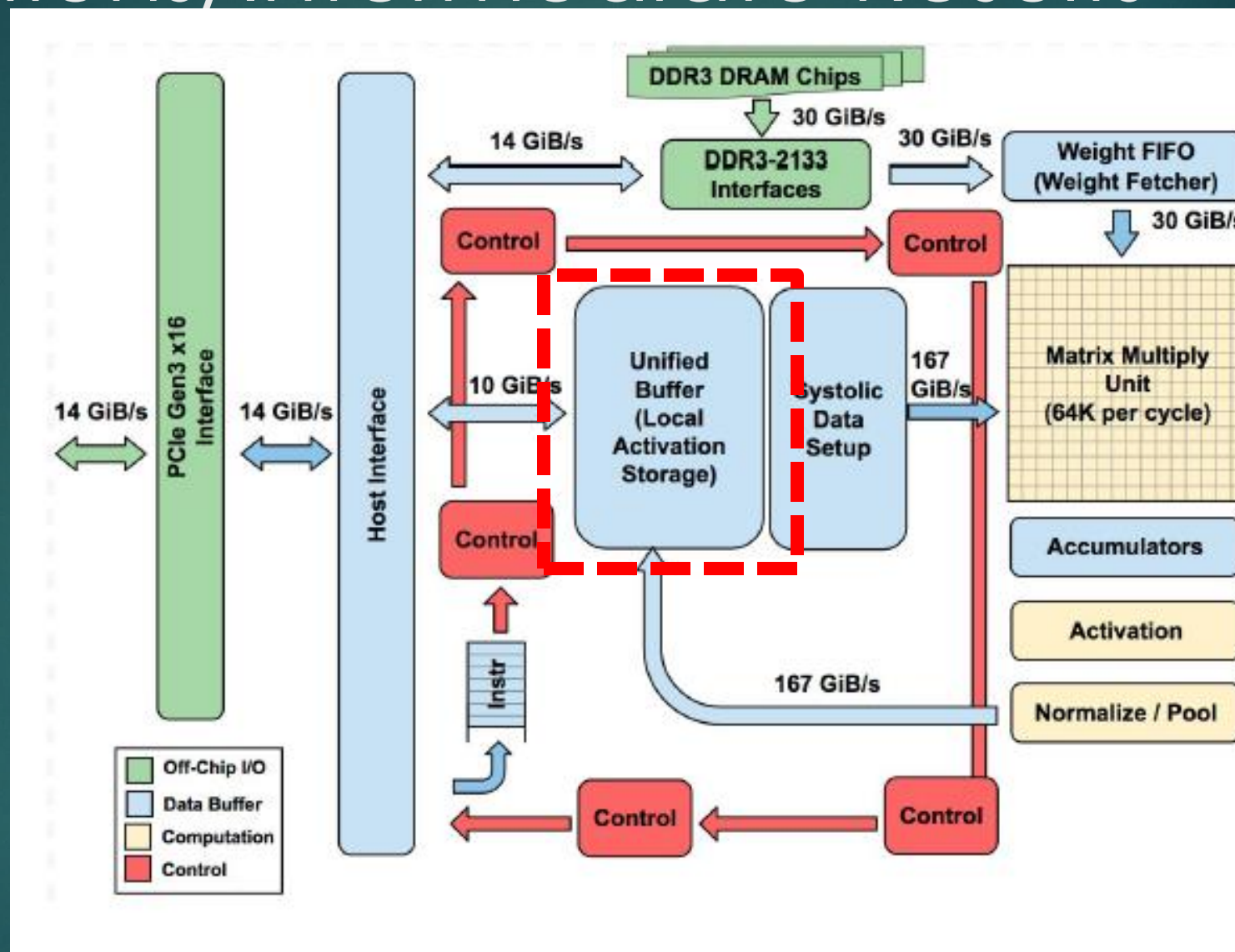
32



8GiB off-chip DRAM, Weight FIFO 64KiB x 4

Activations/Intermediate Results

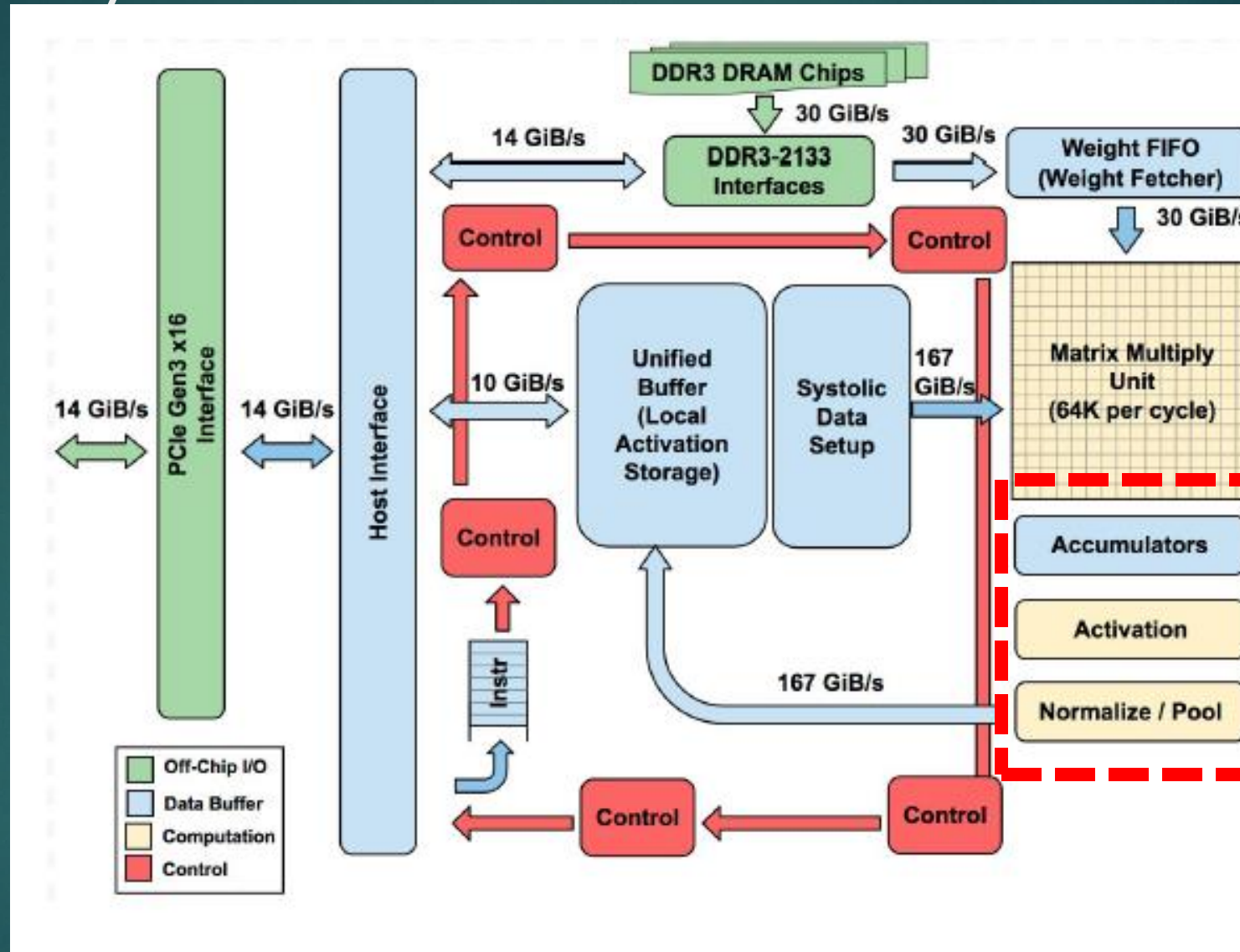
33



Unified Buffer: 24MiB, data from/to host CPU
Connects on 16x PCIE 3.0 through Sata interface

Other Layers

34



Specialized Units for; Activation Functions, Normalization and Pooling

Layout

35

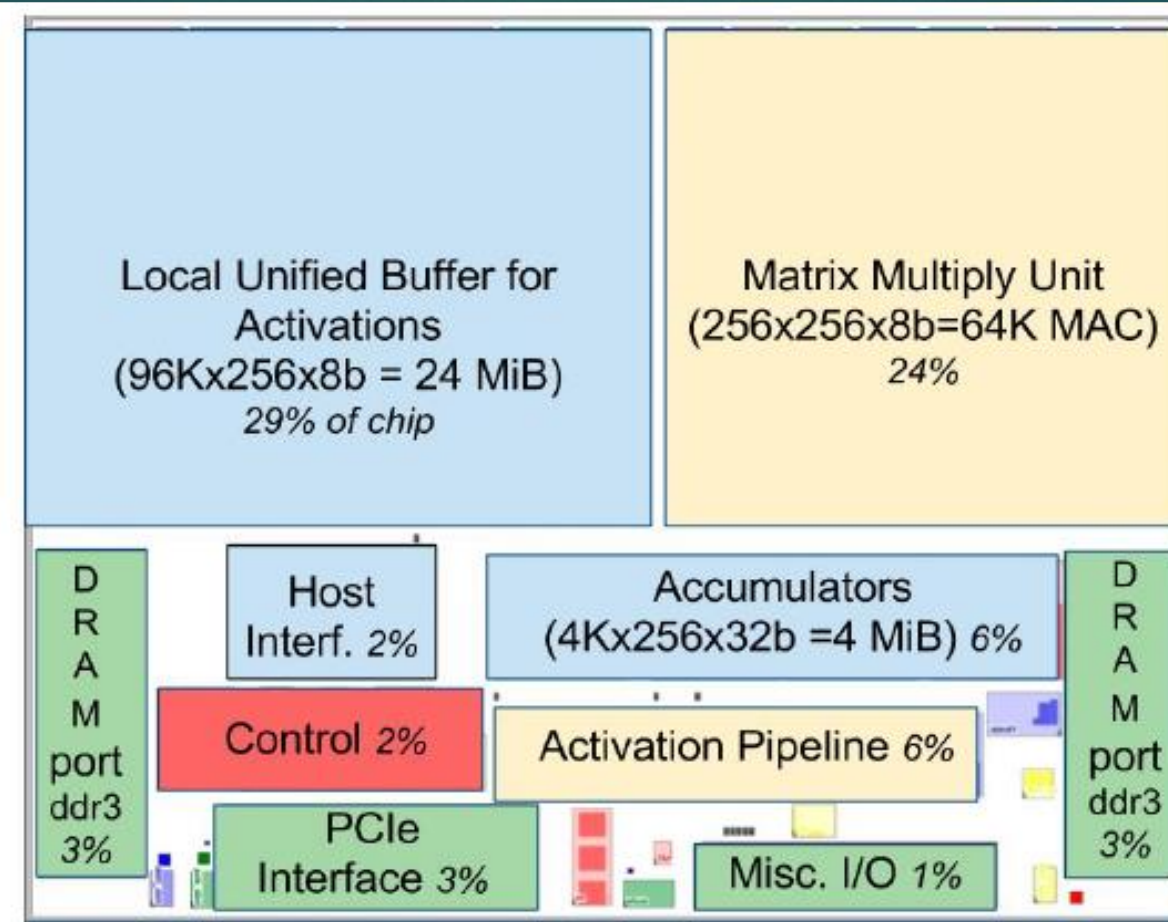


Figure 2. Floor Plan of TPU die. The shading follows Figure 1. The light (blue) data buffers are 37% of the die, the light (yellow) compute is 30%, the medium (green) I/O is 10%, and the dark (red) control is just 2%. Control is much larger (and much more difficult to design) in a CPU or GPU

CISC Instructions

36

- ▶ About a dozen
- ▶ 5 are the most important ones
- ▶ 1. **Read_Host_Memory** reads data from the CPU host memory into the Unified Buffer (UB).
- ▶ 2. **Read_Weights** reads weights from Weight Memory into the Weight FIFO as input to the Matrix Unit.
- ▶ 3. **MatrixMultiply/Convolve** causes the Matrix Unit to perform a matrix multiply or a convolution from the Unified Buffer into the Accumulators.
 - ▶ A matrix operation takes a variable-sized $B \times 256$ input, multiplies it by a 256×256 constant weight input, and produces a $B \times 256$ output, taking B pipelined cycles to complete.

Instructions Contd.

37

- ▶ 4. **Activate** performs the nonlinear function of the artificial neuron, with options for ReLU, Sigmoid, and so on. Its inputs are the Accumulators, and its output is the Unified Buffer. It can also perform the pooling operations needed for convolutions using the dedicated hardware on the die, as it is connected to nonlinear function logic.
- ▶ 5. **Write_Host_Memory** writes data from the Unified Buffer into the CPU host memory.

Design Philosophy

38

- ▶ Keep the Matrix Unit busy
- ▶ 4-stage pipeline
 - ▶ Instructions can occupy pipeline “stages” for thousands of cycles
- ▶ Overlap instructions with Matrix Multiply
- ▶ Read_weights uses access/decoupling

Systolic Array Approach

39

- ▶ Software “thinks” that 256 input data is read once and updates the output accumulators in a single cycle.
- ▶ In reality, the data propagates as a wave, and values accumulate accordingly

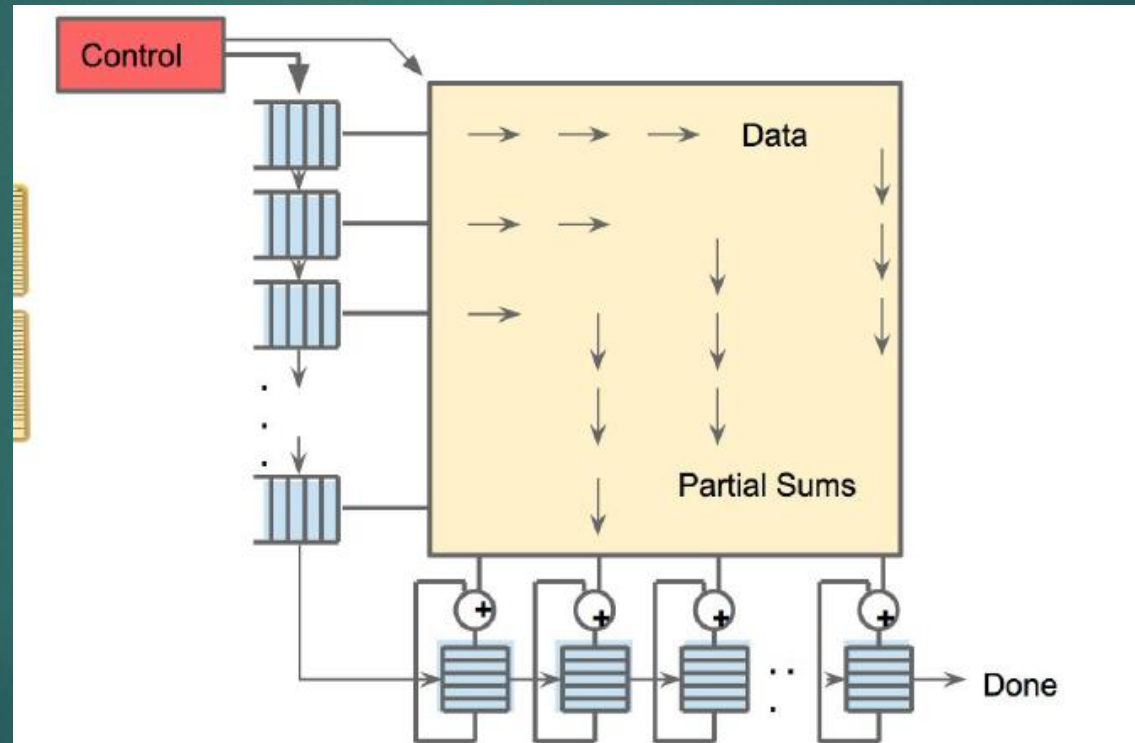
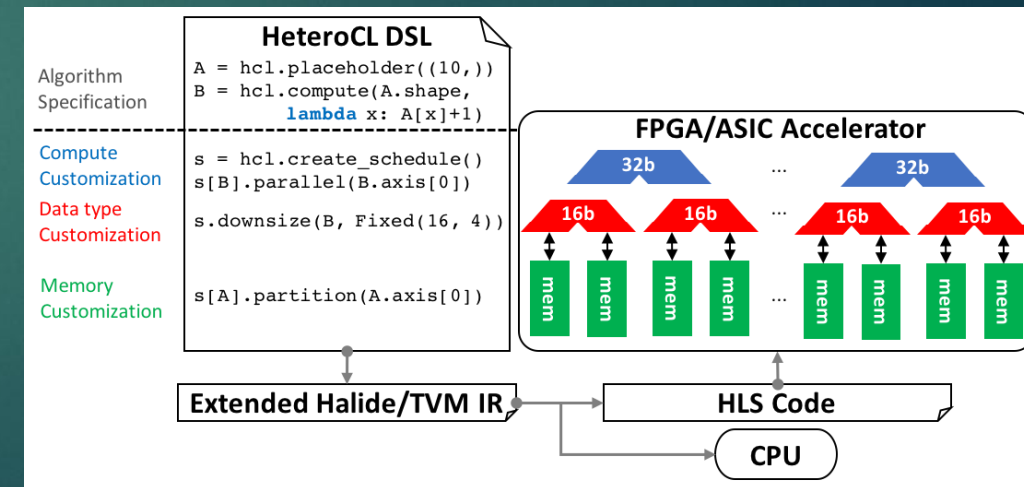
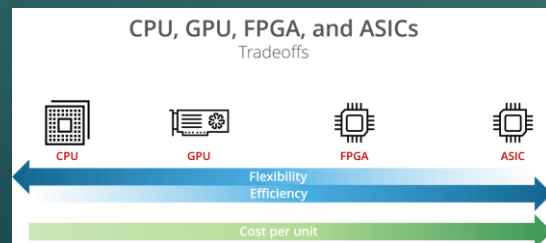
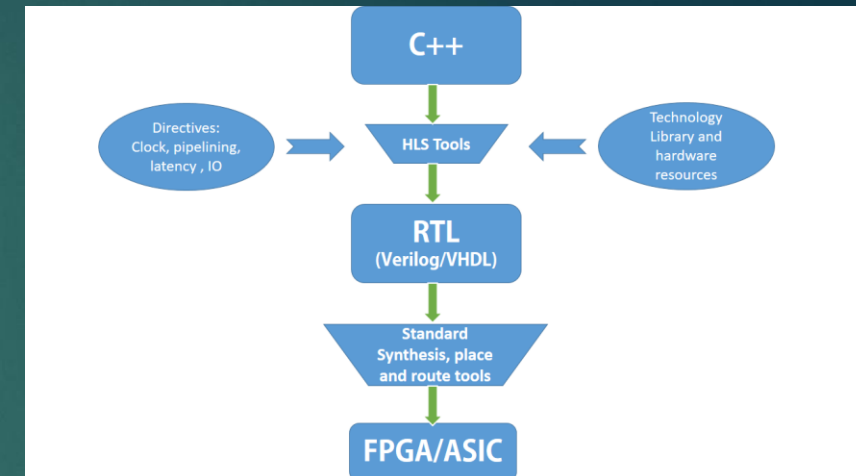


Figure 4. Systolic data flow of the Matrix Multiply Unit. Software has the illusion that each 256B input is read at once, and they instantly update one location of each of 256 accumulator RAMs.

Conclusion

40

- ▶ ASIC => Fix Hardware / Logics for Computing
- ▶ Design for Specific Functions
- ▶ Can Start from High Level Code
- ▶ HLS Tools Compile into Hardware/Logic Code
- ▶ Chips Production
- ▶ Assembly for Accelerator Cards
- ▶ Host Control for Computing
- ▶ TPU => an Example of ASIC Computing



END

QUESTIONS & ANSWERS